Mutation Tester Usability Test

About this test

Thanks for helping us test MutationTester, the mutation testing tool our capstone team built. You'll set it up, run it on a sample Python project, read its report, use it to improve some tests, add your own mutation rule and try its safety check. It takes about 55 minutes, and you'll do it on your own.

What we're checking. Whether someone new to the tool can:

1. Install the tool and run it on a sample project by following the written instructions.
2. Read the HTML report and explain what killed, survived and no coverage mean.
3. Use a surviving mutant to write a test that kills it.
4. Change which operators run.
5. Add a new mutation rule to a spec file without being shown the rule format.
6. Tell when a mutant might be unsafe to run, and decide what to do with it.

How it works. No one from the team will be there, so please:

- Start a screen recording with your microphone on before you begin. Recording yourself in a Teams or Zoom meeting works fine.
- Think out loud the whole time. Say what you're looking at, what you expect to happen and anything that surprises you.
- Do the tasks in order, and use only this doc and the files in the repository. Please don't search online or ask anyone for help.
- Each task has a time limit. If you hit it, or you're stuck on one step for 5 minutes, write down where you got stuck and move on to the next task.
- If setup fails and you can't run the tool at all, stop there and send us what you have.

We're testing the tool, not you. Getting stuck is useful, because it shows us what to fix.

Part	Time
Install and first run (Task 1)	12 min
Tasks 2 to 7	35 min
Feedback questions	5 min
Total	**55 min**

When you're done, stop the recording and send it to us, along with this doc with your answers and Your results filled in.

Setup

Setup is Task 1. Note the time you start, then read the explanation below and follow the steps. It takes about 10 minutes, most of it the setup script installing Java, Maven and Python, plus 1 to 2 minutes for the build.

What mutation testing is. Passing tests don't prove much on their own. A test suite can pass every time and still miss real bugs, because the tests might never check the right things. Mutation testing checks your tests by planting small bugs on purpose and seeing whether the tests catch them.

The tool makes one small change to the code at a time, such as turning `a + b` into `a - b`. Each changed copy of the program is called a *mutant*. The tool runs your tests against every mutant, and each one ends up in one of three states:

- *Killed*: at least one test failed. Your tests noticed the change, which is what you want.
- *Survived*: every test still passed. Your tests ran the changed code but didn't catch the bug, so they probably need a better check.
- *No coverage*: no test ever runs the changed line, so nothing could have caught it.

The share of mutants that get killed is called the *mutation score*. The higher it is, the more kinds of bugs your tests would catch.

Where the changes come from. Each kind of change is called a *mutation operator*, like "turn + into -" or "replace a condition with `true`". You can pick built-in operators in the config file, or write your own rules in a spec file.

Steps (Windows). Open PowerShell as administrator (right-click PowerShell and choose Run as administrator), and run every command there.

1. Get the code and go into its folder:
    - `git clone https://github.com/EllieJin04/mutationtestor.git`
    - `cd mutationtestor`
2. Run the setup script. It checks what's already installed and installs only what's missing (Java, Maven, Python and pytest). It can take several minutes:
    - `powershell -ExecutionPolicy Bypass -File .\chocolatey-setup.ps1 --install --only java,maven,python`
    - When it's done, it should say `All required tools are available.`
3. Load the tools into this window. The dot and space at the start matter, and you'll need to run this again in any new PowerShell window:
    - `. .\demo-env.ps1`
4. Build the tool (takes 1 to 2 minutes): `mvn package -DskipTests`
5. Run it on the Python sample project (takes about 20 seconds):
    - `java -jar target\MutationTester-1.0-SNAPSHOT-jar-with-dependencies.jar samples\PytestProject newPython.config`
6. Run it on the safety demo project. This one may stop and ask you about some mutants. Task 7 explains why; for now, read what it says and make a choice. Its report goes in that project's own folder, so it won't replace the one from step 5:
    a. `cd samples\SafetyDemoProject`
    b. `java -jar ..\..\target\MutationTester-1.0-SNAPSHOT-jar-with-dependencies.jar . safety-demo.config`
    c. `cd ..\..`
7. Open `output\report\mutation-report-demo.html` in a web browser.

On macOS or Linux: in step 2, run `bash setup-dependencies.sh --install` instead, skip step 3, in step 6 run `bash demo-safety.sh` from the repository folder (it prints where it saves its report), and use `/` instead of `\` in the paths.

The config file. `newPython.config` tells the tool what to do. The settings you'll use:

Setting	Meaning
source_code_directory	The folder of code to mutate
mutator_type	Which mutations to make: one or more operator names, or CUSTOM to use a spec file
custom_mutation_spec	The spec file of mutation rules, when mutator_type=CUSTOM
test_command	The command that runs the project's tests

Test tasks

Do the tasks in order, and keep thinking out loud. For each one, note the time you start, do the task and type your answers under it, then write how long it took in Your results. If you hit the time limit, write down where you got stuck and move on.

Task	Time limit	Tests goal
1. Install and run	12 min	1
2. Read the report	5 min	2
3. Cover untested code	7 min	3
4. Investigate a survivor	6 min	2, 3
5. Choose operators	4 min	4
6. Add a rule to the spec	8 min	5
7. Check a risky mutant	5 min	6

Task 1. Install and run

> *Follow the steps in Setup until you have the report open in your browser.*

Was there any step where you weren't sure what to do? Which one, and why?

Your answers:

Task 2. Read the report

> *Look around the report and answer these questions.*

1. How many mutants were killed, and how many had no coverage?
2. Pick one killed mutant. What did it change, and which test caught it?
3. In your own words, what do killed, survived and no coverage mean?

Your answers:

Task 3. Cover untested code

> *The report says the mutant on line 6 of NoCoverageDemo.py has no coverage. Add a test for that function in tests/TestNoCoverage.py, then run the tool again.*

What changed in the report?

Your answers:

Task 4. Investigate a survivor

> *Make a copy of newPython.config and change it to use StatementMutation.txt. Run it, find the mutant that survived, and either write a test that kills it or explain why that isn't possible.*

Which mutant survived? Paste the test you wrote, or explain why no test can kill it. Does survived mean what you said in Task 2?

Your answers:

Task 5. Choose operators

> *Change your config so the tool only turns a + into a -, and nothing else. Then run it.*

What did you change in the config, and how did you figure it out? How many mutants did the run make?

Your answers:

Task 6. Add a rule to the spec

> *This time you'll write your own mutation instead of picking a built-in one. Open StatementMutation.txt, the spec file you used in Task 4. Add a new rule to it that changes every + into a -. Then run the tool again with your config from Task 4, and find your new mutants in the report.*

If you're stuck on the rule format for 3 minutes, look at OperatorMutations.txt, and say so in your answers.

1. Paste the rule you added.
2. Where did you look first to figure out how rules are written?
3. How many runs did it take to get a rule that worked? Did any error message help?
4. Did you need the OperatorMutations.txt hint?
5. From 1 (very hard) to 5 (very easy), how easy was it to add the rule? What would have made it easier?

Your answers:

Task 7. Check a risky mutant

Some code is risky to mutate. If a mutant changes code that writes or deletes files, talks to a database or the network, or starts other programs, running it could do real damage. So before

running each mutant, the tool looks at the change and labels it SAFE, REVIEW, BLOCK or UNKNOWN. SAFE mutants run on their own. For the rest, the tool stops and asks you what to do.

Setup step 6 ran a different sample project that's built to trigger those checks. Its tests only write to temporary folders and never make real network calls or start real programs, so nothing you pick here can cause harm.

> *Open the safety demo's report at samples\SafetyDemoProject\output\report\mutation-report-demo.html and find the mutants you didn't run. If you want to try different choices, run Setup step 6 again.*

1. Pick one prompt. What label did the tool give it, and why did it think the change might be risky?
2. What did you choose at each prompt, and why? Was it clear what each choice would do?
3. Where do the mutants you didn't run show up in the report? Do they count toward the mutation score?
4. From 1 (very unclear) to 5 (very clear), how clear were the safety prompts?

Your answers:

Feedback questions

Answer these right after Task 7. Part 1 is the 10 standard System Usability Scale (SUS) statements, then 5 questions about this tool.

Part 1: System Usability Scale. After each statement, write a rating from 1 (strongly disagree) to 5 (strongly agree).

1. I think that I would like to use this tool frequently.
2. I found the tool unnecessarily complex.
3. I thought the tool was easy to use.
4. I think that I would need the support of a technical person to be able to use this tool.
5. I found the various functions in this tool were well integrated.
6. I thought there was too much inconsistency in this tool.
7. I would imagine that most people would learn to use this tool very quickly.
8. I found the tool very cumbersome to use.
9. I felt very confident using the tool.
10. I needed to learn a lot of things before I could get going with this tool.

Part 2: About this tool. Type your answer after each question.

1. Which step of the instructions was hardest, and why?
2. Was it clear which line of code each mutant changed? What would have made it clearer?
3. How confident are you that the test you wrote in Task 3 was a good one? (1 to 5)
4. What would you need before using this tool on your own project?
5. Anything else you'd change?

Your results

Fill this in as you go: how long each task took, whether you finished it, and anything that confused you, in your own words.

Name: · **Date:** · **Operating system:**

Task	Time taken (min)	Result	Where you got stuck or weren't sure what to do	Anything else you noticed
1. Install and run		[Result]		
2. Read the report		[Result]		
3. Cover untested code		[Result]		
4. Investigate a survivor		[Result]		
5. Choose operators		[Result]		
6. Add a rule to the spec		[Result]		
7. Check a risky mutant		[Result]		